

Transpilador Python a Java (Analizador Léxico y Sintáctico)

Proyecto de Programación de Sistemas Base 1

Materia	Programación de Sistemas Base 1
Institución	Universidad Autónoma de Tamaulipas
Semestre	2026-1
Profesor(es)	Muñoz Quintero Dante Adolfo
Integrantes	Suarez Martinez Maciel Francisco Estrada Olvera Frank Ortega Resendiz Luis Fernando
Repositorio	https://github.com/NEUTRALDUST21/Proyecto Integrador Base 1 Equipo S.W.A.T. 2026-1.git
Fecha	17 de mayo de 2026

1. Introducción

El análisis léxico y sintáctico son las primeras fases de un compilador o intérprete. En este proyecto se diseña e implementa un transpilador funcional capaz de convertir código fuente escrito en Python a código Java, realizando el análisis de los tokens y estructurando la gramática correspondiente.

Comprender esta etapa de forma práctica es fundamental en la formación de un ingeniero en sistemas computacionales que desee entender la construcción de herramientas de procesamiento de lenguajes. Este transpilador incluye un analizador léxico que gestiona la indentación estricta de Python y un analizador sintáctico que genera bloques equivalentes en Java.

2. Objetivos

Objetivo general

Diseñar e implementar un analizador léxico y sintáctico (transpilador) en Python que transforme programas fuente básicos de Python en su equivalente estructurado en Java, con un manejo preciso de errores léxicos y de indentación.

Objetivos específicos

1. Definir la especificación del subconjunto de Python soportado y su equivalencia en Java.
2. Establecer un catálogo de tokens que incluya palabras reservadas, operadores, números, cadenas y niveles de indentación.
3. Implementar el analizador léxico usando Python, asegurando el reconocimiento correcto de los bloques (INDENT y DEDENT).
4. Implementar un generador de código que infiera tipos básicos (int, double, boolean, String) en Java.

3. Especificación del Lenguaje

3.1 Nombre y propósito

El transpilador maneja un subconjunto de Python orientado a cálculos aritméticos, asignaciones, condicionales (if/elif/else) y ciclos (while). El código resultante es un programa Java dentro de la clase principal Main.

3.2 Catálogo de tokens

Categoría	Ejemplos	Descripción
KEYWORD	if, else, elif, while, print, True, False	Palabras reservadas del lenguaje
IDENTIFIER	x, total, contador	Nombres de variables o

		funciones
NUMBER	10, 3.14, 0	Números enteros o flotantes
STRING	"hola mundo"	Cadenas de texto
OPERATOR	+, -, *, /, %, =, ==, >, <, !=	Operadores matemáticos y lógicos
INDENT / DEDENT	(Espacios al inicio de línea)	Controlan la estructura de bloques de código

4. Implementación

A continuación se presentan fragmentos clave de la implementación. El código completo se encuentra en el repositorio GitHub.

4.1 Definición de tokens – token_types.py

```
class TokenType(Enum):
    # Palabras reservadas de Python
    IF = auto()
    ELSE = auto()
    ELIF = auto()
    WHILE = auto()
    FOR = auto()
    IN = auto()
    DEF = auto()
    RETURN = auto()
    PRINT = auto()
    TRUE = auto()
    FALSE = auto()
    NONE = auto()
    AND = auto()
    OR = auto()
    NOT = auto()

    # Identificadores y literales
    IDENTIFIER = auto()
    NUMBER = auto()
    STRING = auto()

    # Operadores
    ASSIGN = auto()
    EQ = auto()
    NE = auto()
    LT = auto()
    LE = auto()
    GT = auto()
    GE = auto()
    PLUS = auto()
    MINUS = auto()
    TIMES = auto()
    DIVIDE = auto()
```

```

MOD = auto()

# Delimitadores
LPAREN = auto()
RPAREN = auto()
LBRACE = auto()
RBRACE = auto()
COLON = auto()
COMMA = auto()
SEMICOLON = auto()
NEWLINE = auto()
INDENT = auto()
DEDENT = auto()

COMMENT = auto()
SPACE = auto()
EOF = auto()
ERROR = auto()

```

4.2 Analizador Léxico (Manejo de Indentación)

Uno de los mayores retos fue el control de la indentación de Python para traducirlo a llaves en Java.

```

def handle_indentation(self):
    col = 1
    while self.peek() == ' ':
        self.advance()
        col += 1

    if col > self.indent_stack[-1]:
        self.indent_stack.append(col)
        self.tokens.append(Token(TokenType.INDENT, "", self.line,
col))
    elif col < self.indent_stack[-1]:
        while col < self.indent_stack[-1]:
            self.indent_stack.pop()
            self.tokens.append(Token(TokenType.DEDENT, "",
self.line, col))

```

4.3 Analizador Sintáctico y Generación de Código Java

```

def parse_assignment(self):
    ident = self.consume(TokenType.IDENTIFIER)
    self.consume(TokenType.ASSIGN, "Expected '='")
    expr = self.parse_expression()

    # Inferencia básica de tipos
    java_type = self.infer_type(expr)
    self.java_code.append(self.indent() + f"{java_type}
{ident.value} = {expr};
")

```

5. Demostración Funcional

5.1 Ejecución con programa válido (entrada.py)

Código de entrada (Python):

```
x = 10
y = 20
if x < y:
    print("x es menor que y")
else:
    print("x no es menor que y")
```

Código generado (Java):

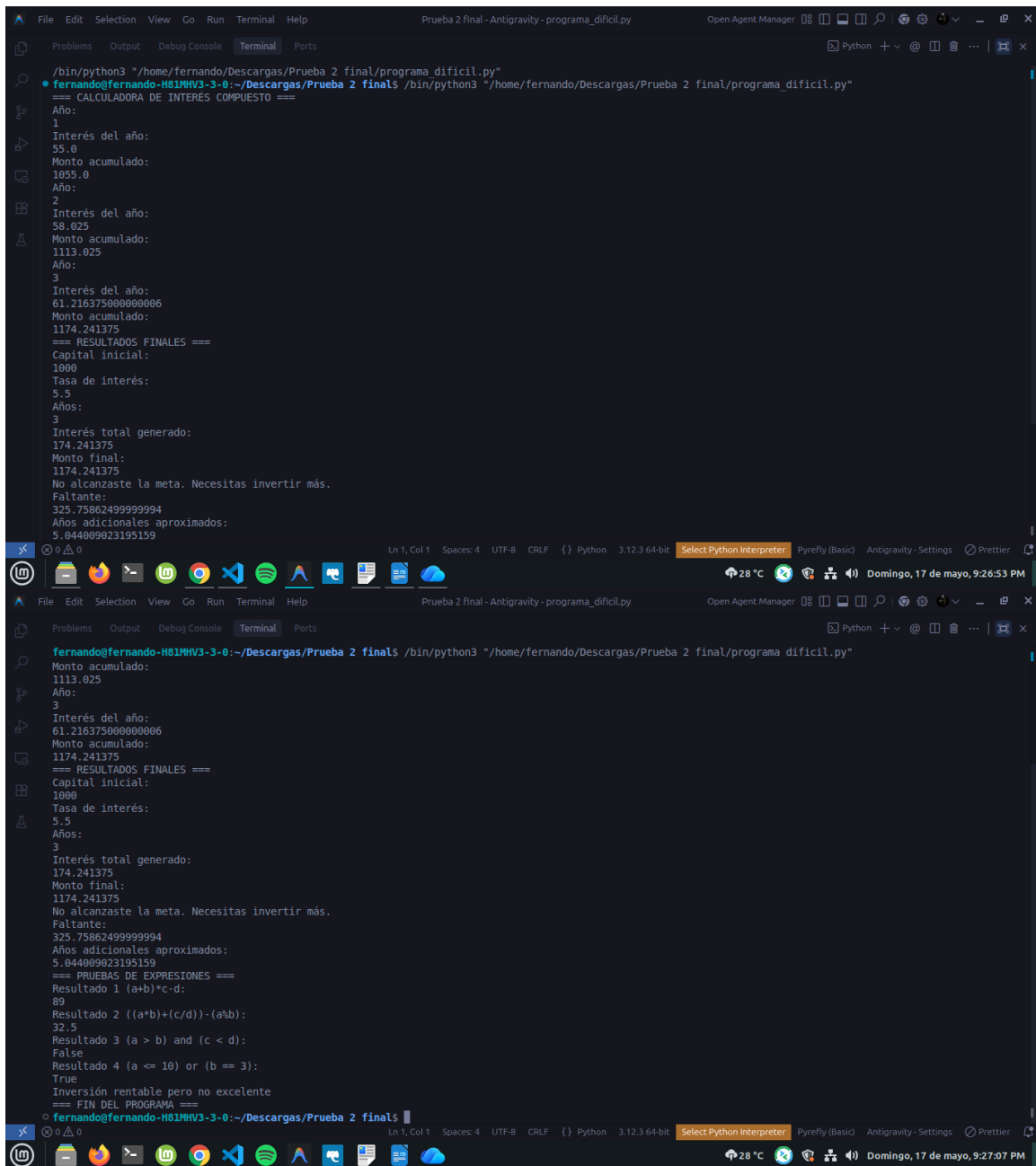
```
public class Main {
    public static void main(String[] args) {
        int x = 10;
        int y = 20;
        if ((x < y)) {
            System.out.println("x es menor que y");
        }
        else {
            System.out.println("x no es menor que y");
        }
    }
}
```



```
/bin/python3 "/home/fernando/Descargas/Prueba 2 final/main.py"
fernando@fernando-H81MHV3-3-0:~/Descargas/Prueba 2 final$ /bin/python3 "/home/fernando/Descargas/Prueba 2 final/main.py"
=====
TRANSPILADOR PYTHON -> JAVA
=====
Ingresa la ruta del archivo .py o .txt: /home/fernando/Descargas/Prueba 2 final/entrada.py
Leyendo archivo: /home/fernando/Descargas/Prueba 2 final/entrada.py
=====
=== TOKENS GENERADOS ===
Token(IDENTIFIER, 'x', line=2, col=1)
Token(ASSIGN, '=', line=2, col=3)
Token(NUMBER, 10, line=2, col=5)
Token(IDENTIFIER, 'y', line=3, col=1)
Token(ASSIGN, '=', line=3, col=3)
Token(NUMBER, 20, line=3, col=5)
Token(IF, 'if', line=4, col=1)
Token(IDENTIFIER, 'x', line=4, col=4)
Token(LT, '<', line=4, col=6)
Token(IDENTIFIER, 'y', line=4, col=8)
Token(COLON, ':', line=4, col=9)
Token(PRINT, 'print', line=5, col=5)
Token(LPAREN, '(', line=5, col=10)
Token(STRING, 'x es menor que y', line=5, col=11)
Token(RPAREN, ')', line=5, col=29)
Token(ELSE, 'else', line=6, col=1)
Token(COLON, ':', line=6, col=5)
Token(PRINT, 'print', line=7, col=5)
Token(LPAREN, '(', line=7, col=10)
Token(STRING, 'x no es menor que y', line=7, col=11)
Token(RPAREN, ')', line=7, col=32)
Token(WHILE, 'while', line=9, col=1)
Token(IDENTIFIER, 'x', line=9, col=7)
Token(GT, '>', line=9, col=9)
Token(NUMBER, 0, line=9, col=11)
Token(COLON, ':', line=9, col=12)
Token(NUMBER, 0, line=9, col=11)
Token(COLON, ':', line=9, col=12)
Token(PRINT, 'print', line=10, col=5)
Token(LPAREN, '(', line=10, col=10)
Token(IDENTIFIER, 'x', line=10, col=11)
Token(RPAREN, ')', line=10, col=12)
Token(IDENTIFIER, 'x', line=11, col=5)
Token(ASSIGN, '=', line=11, col=7)
Token(IDENTIFIER, 'x', line=11, col=9)
Token(MINUS, '-', line=11, col=11)
Token(NUMBER, 1, line=11, col=13)
Token(EOF, '', line=11, col=14)
=====
CÓDIGO JAVA GENERADO EXITOSAMENTE
Archivo: /home/fernando/Descargas/Prueba 2 final/entrada.java
=====
=== CÓDIGO JAVA GENERADO ===
public class Main {
    public static void main(String[] args) {
        int x = 10;
        int y = 20;
        if ((x < y)) {
            System.out.println("x es menor que y");
        } else {
            System.out.println("x no es menor que y");
        }
        while ((x > 0)) {
            System.out.println(x);
            int x = (x - 1);
        }
    }
}
```

5.2 Ejecución del programa complejo

El transpilador maneja correctamente operaciones anidadas y ciclos avanzados como se demostró en "programa_dificil.py", infiriendo los tipos a int y double, y manejando expresiones booleanas complejas con operadores relacionales y lógicos.



```
/bin/python3 "/home/fernando/Descargas/Prueba 2 final/programa_dificil.py"
fernando@fernando-H81MHV3-3-0:~/Descargas/Prueba 2 final$ /bin/python3 "/home/fernando/Descargas/Prueba 2 final/programa_dificil.py"
=== CALCULADORA DE INTERES COMPUESTO ===
Año:
1
Interés del año:
55.0
Monto acumulado:
1055.0
Año:
2
Interés del año:
58.025
Monto acumulado:
1113.025
Año:
3
Interés del año:
61.216375000000006
Monto acumulado:
1174.241375
=== RESULTADOS FINALES ===
Capital inicial:
1000
Tasa de interés:
5.5
Años:
3
Interés total generado:
174.241375
Monto final:
1174.241375
No alcanzaste la meta. Necesitas invertir más.
Faltante:
325.75862499999994
Años adicionales aproximados:
5.044009023195159
=== PRUEBAS DE EXPRESIONES ===
Resultado 1 (a+b)*c-d:
89
Resultado 2 ((a*b)+(c/d))-(a*b):
32.5
Resultado 3 (a > b) and (c < d):
False
Resultado 4 (a <= 10) or (b == 3):
True
Inversión rentable pero no excelente
=== FIN DEL PROGRAMA ===
fernando@fernando-H81MHV3-3-0:~/Descargas/Prueba 2 final$
```

6. Conclusiones

- Luis Fernando Ortega Resendiz: Diseñar este transpilador me permitió comprender en la práctica cómo funciona internamente un lenguaje, separar la lógica léxica de la sintáctica fue el mayor reto pero el mejor aprendizaje.
- Maciel Francisco Suarez Martinez / Frank Estrada Olvera: El manejo de la indentación en Python para transformarla a bloques con llaves en Java fue la parte más desafiante de implementar el Lexer.

7. Referencias

- [1] Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2007). Compilers: Principles, Techniques, and Tools (2.a ed.). Pearson Addison-Wesley.
- [2] Python Software Foundation. Documentación oficial de Python.